Digital Filters: Introduction

Signal Separation, Signal Restoration, and the Language of Filters

Why Filters Matter

- Filters are one of the most important tools in DSP.
- Their outstanding performance is a major reason DSP has become so widely used.
- Filters serve **two** general purposes:
  1. **Signal separation**
  2. **Signal restoration**

Use 1: Signal Separation

Needed when a signal is contaminated by interference, noise, or other signals.

Example: A device measuring a baby's electrical heart activity (EKG) while still in the womb.

- The raw signal is corrupted by the mother's breathing and heartbeat.
- A filter separates these signals so each can be analyzed individually.

Use 2: Signal Restoration

Needed when a signal has been distorted in some way.

Examples:

- An audio recording made with poor equipment $\rightarrow$ filtered to better represent the true sound.
- An image acquired with an improperly focused lens, or a shaky camera $\rightarrow$ deblurring.

Analog vs. Digital Filters

Both signal separation and restoration problems can be solved with **analog** or **digital** filters.

Analog Filters

- Cheap
- Fast
- Large dynamic range (amplitude & frequency)
- Limited by resistor/capacitor accuracy & stability

Digital Filters

- Vastly superior performance
- Example: low-pass filter (Ch. 16) – gain 1 ± 0.0002 up to 1000 Hz, < 0.0002 above 1001 Hz
- Transition band: only 1 Hz wide!
- Can be thousands of times better than analog

Because digital filters are so good, the focus shifts from *electronics limitations* to the *theoretical processing of signals*.

The "Time Domain" – More General Than It Sounds

- Filter input/output signals are usually described as being in the **time domain**, since signals are typically sampled at regular time intervals.
- But sampling can also occur over other domains – most commonly **space**.

Example: Strain sensors mounted every 1 cm along an aircraft wing, sampled simultaneously.

Key takeaway

"Time domain" in DSP may literally mean samples over time, *or* it may just be a general term for whatever domain the samples were taken in.

Three Faces of a Linear Filter (Fig. 14-1)

Every linear filter can be fully described by **three** equivalent representations:

- **Impulse response**
- **Step response**
- **Frequency response**

- Each contains the *complete* information about the filter, just in a different form.
- Specify **any one** of the three $\rightarrow$ the other two are fixed and can be calculated directly.
- Together, they describe how the filter reacts under different circumstances (see **Fig. 14-1**).

Implementing Filters: Convolution (FIR)

- The most direct way to implement a digital filter: **convolve** the input signal with the filter's impulse response.
- *All* possible linear filters can be made this way.
- Each output sample = weighted sum of input samples.
- The impulse response used this way is called the **filter kernel**.
- Filters made by convolution are called **Finite Impulse Response (FIR)** filters.

Implementing Filters: Recursion (IIR)

- **Recursion** is another way to build digital filters (detailed in Chapter 19).
- Output samples depend on *previous output values* as well as input values.
- Defined by **recursion coefficients** instead of a filter kernel.
- To find a recursive filter's impulse response: feed in an impulse and observe the output.
- These responses are sinusoids that **decay exponentially** – in principle infinitely long.
- In practice, once the amplitude drops below the system's round-off noise, remaining samples can be ignored.

Terminology

Recursive filters $\rightarrow$ **Infinite Impulse Response (IIR)** filters, in contrast to **FIR** (convolution-based) filters.

Finding the Step Response

- **Step response** = output when the input is a step (also called an *edge*, or *edge response*).
- Since a step is the **integral** of an impulse, the step response is the **integral** of the impulse response.

Two ways to find it:

1. Feed a step waveform into the filter and observe the output.
2. Integrate the impulse response.

Note: true integration applies to continuous signals; for discrete signals, a **running sum** (discrete integration) is used instead.

Finding the Frequency Response

- Found by taking the **DFT** (via the FFT algorithm) of the impulse response.
- Can be plotted two ways:
  - **Linear scale** – best for showing passband ripple and roll-off.
  - **Decibel (log) scale** – needed to show stopband attenuation.

Decibels: A Quick Review

- Named after Alexander Graham Bell.
- **1 bel** = power changed by a factor of **10**.
- **1 decibel (dB)** = one-tenth of a bel.

dB	-20	-10	0	10	20
Power ratio	0.01	0.1	1	10	100

Rule of thumb

Every **10 dB** $\rightarrow$ power changes by a factor of **10**.

Decibels: Power vs. Amplitude

- Amplitude is proportional to the **square root** of power.
- Example: an amplifier with **20 dB** of gain $\rightarrow$ power increases by $100\times$, but amplitude only increases by $10\times$.

Rule of thumb

Every **20 dB** $\rightarrow$ amplitude changes by a factor of **10**.

Formulas (base-10 log):

$$dB = 10 \log_{10}\left(\frac{P_2}{P_1}\right), \qquad dB = 20 \log_{10}\left(\frac{A_2}{A_1}\right)$$

Using natural log (when only ln is available):

$$dB = 4.342945 \ln\left(\frac{P_2}{P_1}\right), \qquad dB = 8.685890 \ln\left(\frac{A_2}{A_1}\right)$$

Referencing a Single Signal in dB

- Decibels normally express the **ratio** between two signals – ideal for describing a system's **gain** (output vs. input).
- Engineers also use dB to describe the amplitude/power of a **single** signal, by referencing it to a standard:
  - **dBV** – referenced to a 1 volt rms signal.
  - **dBm** – referenced to a signal producing 1 mW into a 600 Ω load ($\approx$ 0.78 V rms).

Key Facts to Remember

The two most important decibel facts

1. -3 **dB** means amplitude drops to **0.707** (power drops to **0.5**).
2. Memorize the standard conversions between decibels and amplitude ratios.

Next: The most popular digital filters, compared chapter by chapter.

Two Kinds of Information

- The **most important part** of any DSP task: understanding how information is contained in the signals you work with.
- Manmade signals use many schemes: AM, FM, single-sideband, pulse-code modulation, pulse-width modulation, etc.

For *naturally occurring* signals, only two are common

1. Information represented in the **time domain**
2. Information represented in the **frequency domain**

Information in the Time Domain

- Describes **when** something occurs and **what the amplitude** of the occurrence is.

Example: Measuring the light output of the sun, sampled once per second.

- Each sample shows what is happening *at that instant*, and its level.
- A solar flare's timing, duration, and development are all directly visible in the signal.

Key point

Each sample is interpretable **on its own**, without reference to any other sample. Even a single sample tells you something meaningful. This is the **simplest** way information can be contained in a signal.

Information in the Frequency Domain

- A more **indirect** form of representation.
- Many things in the universe show **periodic motion**: a struck wine glass rings; a grandfather clock's pendulum swings; stars and planets rotate and revolve.
- Measuring the **frequency, phase, and amplitude** of periodic motion reveals information about the system producing it.

Example: The sound of a ringing wine glass.

- Its fundamental frequency and harmonics relate to the mass and elasticity of the glass.

Key point

A **single sample**, by itself, carries **no information** about the periodic motion. The information lives in the **relationship between many points** in the signal.

Why This Matters: Step vs. Frequency Response

- **Step response** $\rightarrow$ shows how **time-domain** information is modified by a system.
- **Frequency response** $\rightarrow$ shows how **frequency-domain** information is modified by a system.

Critical trade-off in filter design

A filter **cannot** be optimized for both domains at once.

- Good time-domain performance $\Rightarrow$ poor frequency-domain performance.
- Good frequency-domain performance $\Rightarrow$ poor time-domain performance.

Matching the Response to the Application

Example: EKG Noise Removal

- Information is in the **time domain**
- **Step response** $\rightarrow$ important
- Frequency response $\rightarrow$ little concern

Example: Hearing Aid Filter

- Information is in the **frequency domain**
- **Frequency response** $\rightarrow$ all-important
- Step response $\rightarrow$ doesn't matter

What makes a filter **optimal** for time-domain or frequency-domain applications?

Why the Step Response?

- Step, impulse, and frequency responses all contain **identical information** — just arranged differently.
- So why focus on the step response instead of the impulse response?
- Answer: it matches how the **human mind** processes signals.
- The step response describes how a filter modifies the "dividing lines" the mind naturally looks for in data.

Segmenting a Signal

- Given an unknown signal, the mind automatically divides it into regions of similar characteristics:
  - smooth regions
  - regions with large amplitude peaks
  - noisy regions
- This segmentation works by identifying the **points that separate** the regions.
- The step function is the purest way to represent such a division — marking where an event starts or ends.
- Mental model: time-domain information $\approx$ a group of step functions dividing the signal into regions.

Risetime

Reference: Fig. 14-2 (a) & (b)

- To distinguish events, the step response duration must be shorter than the spacing between events.
- The step response should therefore be as **fast** as possible.
- Risetime is usually specified as the number of samples between the 10% and 90% amplitude levels.

Why isn't a fast risetime always possible?

- Noise reduction requirements
- Inherent limitations of the data acquisition system
- Avoiding aliasing

Overshoot

Reference: Fig. 14-2 (c) & (d)

- Overshoot must generally be **eliminated**.
- It changes the amplitude of samples — a basic distortion of the signal's information.

Key Question

Is the overshoot you observe coming from the thing you are trying to measure, or from the filter you have used?

Symmetry and Linear Phase

Reference: Fig. 14-2 (e) & (f)

- Often desired: the upper half of the step response is symmetrical with the lower half.
- This symmetry makes rising edges look the same as falling edges.
- Called **linear phase**, because the corresponding frequency response has a phase that is a straight line.

Risetime, Overshoot, and Symmetry — the three keys to evaluating time domain filters.

The Four Basic Frequency Responses

Reference: Fig. 14-3

- **Passband**: frequencies that are passed unaltered
- **Stopband**: frequencies that are blocked
- **Transition band**: region between passband and stopband
- **Cutoff frequency**: division between passband and transition band

- Analog filters: cutoff usually defined where amplitude drops to 0.707 (i.e., $-3\,\text{dB}$).
- Digital filters: less standardized — 99%, 90%, 70.7%, or 50% levels are all common definitions.

Measuring Filter Performance

Reference: Fig. 14-4

- **Roll-off** (a, b) — a fast roll-off (narrow transition band) is needed to separate closely spaced frequencies.
- **Passband ripple** (c, d) — passband frequencies should move through the filter unaltered, with no ripple.
- **Stopband attenuation** (e, f) — stopband frequencies must be adequately blocked.

What About Phase?

- Phase is usually absent from these frequency-domain parameters. Why?
- **Phase often isn't important**: e.g., the phase of an audio signal is nearly random and carries little useful information.
- When phase *is* important, digital filters can easily achieve a **perfect (zero) phase response** — all frequencies pass with zero phase shift.
- Analog filters, by comparison, perform poorly in this respect.

Review: From Impulse Response to Frequency Response

- The FFT converts an N-sample filter kernel into an N-point real part and an N-point imaginary part.
- Only samples 0 to $N/2$ carry useful information; the rest are duplicate negative frequencies.
- Converting to polar notation gives **magnitude** and **phase**, each with $N/2 + 1$ samples.

Example: a 256-point impulse response $\rightarrow$ frequency response from point 0 to 128.

- Sample 0 = DC (zero frequency)
- Sample 128 = one-half of the sampling rate

Zero-Padding and Frequency Resolution

- The FFT requires a power-of-two length. An 80-point kernel needs 48 zeros added to reach 128 samples.
- Padding with zeros **does not change** the impulse response — the added zeros vanish during convolution.
- Padding further (256, 512, 1024 points, . . .) gives **closer spacing** of points in the frequency response.
- In the limit of infinite zero-padding, the frequency response becomes a **continuous curve** between DC and half the sampling rate.
- The DFT output is simply a sampling of this continuous line.

Trade-offs in Practice

- The "good" and "bad" parameters discussed are only **generalizations** — many signals don't fit neatly into these categories.
- **Example:** an EKG signal contaminated with 60 Hz interference.
  - The information of interest lies in the time domain.
  - The interference is best handled in the frequency domain.
- The best design involves trade-offs and may go against conventional wisdom.

Remember

A paragraph in a book doesn't give you a license to stop thinking.

Starting from Low-Pass

- High-pass, band-pass, and band-reject filters are all designed by starting with a **low-pass filter** and converting it.
- This is why most filter design discussions only give low-pass examples.
- Two methods exist for low-pass $\rightarrow$ high-pass conversion:
  - **Spectral inversion**
  - **Spectral reversal**
- Both are equally useful.

Spectral Inversion: The Recipe

Reference: Fig. 14-5

- Start with a low-pass windowed-sinc kernel (Chapter 16), e.g., 51 points long.
- Its frequency response is found by zero-padding and taking an FFT.

Two steps to convert low-pass $\rightarrow$ high-pass:

1. Change the sign of every sample in the filter kernel.
2. Add one to the sample at the **center of symmetry**.

- Effect: the frequency response is flipped **top-for-bottom**.
- Passbands $\leftrightarrow$ stopbands (low-pass $\leftrightarrow$ high-pass, band-pass $\leftrightarrow$ band-reject).

Why Spectral Inversion Works

Reference: Fig. 14-6

- Input $x[n]$ is applied to two parallel systems:
  - A low-pass filter with impulse response $h[n]$
  - An all-pass system with impulse response $\delta[n]$ (does nothing)
- Output: $y[n] =$ (all-pass output) $-$ (low-pass output).
- Subtracting the low-frequency content leaves only **high-frequency components** $\rightarrow$ a high-pass filter.

- Parallel systems with added (subtracted) outputs combine into a single kernel:

$$\text{High-pass kernel} = \delta[n] - h[n]$$

- i.e., negate all samples, then add one at the center of symmetry.

Restrictions on Spectral Inversion

- The method relies on low-frequency components from the low-pass filter and the all-pass system having the **same phase**, so subtraction fully cancels them.

This imposes two requirements:

1. The original filter kernel must have left–right symmetry (zero or linear phase).
2. The impulse must be added exactly at the **center of symmetry**.

Spectral Reversal

Reference: Fig. 14-7

- Alternative method to spectral inversion.
- High-pass kernel (c) is formed by changing the sign of **every other sample** of the low-pass kernel (a).
- Effect: flips the frequency domain **left-for-right** — 0 becomes 0.5, and 0.5 becomes 0.

Example:

- Low-pass cutoff frequency: 0.15
- Resulting high-pass cutoff frequency: 0.35

Why Spectral Reversal Works

- Changing the sign of every other sample $\equiv$ multiplying the kernel by a sinusoid of frequency 0.5.
- This shifts the frequency domain by 0.5 (Chapter 10).

- Consider the negative frequencies between -0.5 and 0 in (b) — mirror images of the frequencies between 0 and 0.5.
- The response in (d) is simply these negative frequencies from (b), shifted by 0.5.

Building Band-Pass and Band-Reject Filters

Reference: Figs. 14-8 and 14-9

- Low-pass and high-pass kernels can be combined:

Combination Rules

- **Adding** the kernels $\rightarrow$ band-reject filter
- **Convolving** the kernels $\rightarrow$ band-pass filter

- Based on how parallel and cascaded systems combine (Chapter 7).

Combining Techniques

- Multiple combination techniques can be used together.
- **Example:** to design a band-pass filter,
  1. Add two filter kernels to form a band-*reject* filter, then
  2. Apply spectral inversion or spectral reversal to convert it to band-*pass*.

All these techniques work very well, with few surprises.